

# SLIDE 1 — What is an Out-of-Bounds Error?

## Definition:

An **out-of-bounds error** occurs when we try to access an array using an **invalid index**.

## Simple idea

**Array Size** → **Check Index** → **Valid?** → **Access**

## Example:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
printf("%d", numbers[5]);
```

## Explanation:

The valid indexes are **0 to 4**.

```
Index: 0  1  2  3  4  
       [10] [20] [30] [40] [50]
```

```
numbers[5] ❌
```

`numbers[5]` is outside the array.

⚠️ **Remember: Invalid index → Out-of-bounds**

---

# SLIDE 2 — Valid vs Invalid Index

For an array of size **5**:

```
int numbers[5];
```

## Valid indexes

```
0  1  2  3  4
```



### Invalid indexes

-1      5      6  
✗      ✗      ✗

### ★ Remember

● Valid → 0 to size - 1

● Invalid → Less than 0 or greater than/equal to size

---

## SLIDE 3 — Why is it Dangerous?

C does **not automatically check array bounds**.

An invalid index may access memory that does not belong to the array.

### Example

```
int numbers[3] = {10, 20, 30};
```

```
printf("%d", numbers[5]);
```

### Possible result:

Unexpected value ✗

The program may produce an unexpected result or behave unpredictably.

### 🧠 Simple Analogy

**3 lockers** → 0, 1, 2

Trying to open **locker 5** means you're accessing something that doesn't exist.

**Array** → **Check the index before accessing!**

---

## SLIDE 4 — How to Stay Safe

Always make sure the index is within the valid range.

### Example

```
int numbers[5] = {10, 20, 30, 40, 50};  
int i = 3;
```

```
if (i >= 0 && i < 5) {  
    printf("%d", numbers[i]);  
}
```

### Explanation:

$i \geq 0 \rightarrow \text{TRUE}$

$i < 5 \rightarrow \text{TRUE}$



Safe to access 

### Using a loop

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", numbers[i]);  
}
```

The loop automatically uses:

**0 → 1 → 2 → 3 → 4**

### Basic Pattern

**CHECK INDEX → THEN ACCESS**

---

## SLIDE 5 — RECAP

### Stay Inside the Array!

#### Flow

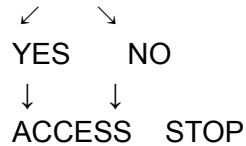
ARRAY SIZE



CHECK INDEX



VALID?



## 🎯 5 Things to Remember

1. Array indexes start from 0
2. Last valid index = size - 1
3. Invalid indexes cause out-of-bounds access
4. Always check the index before accessing
5. Use loops carefully with array limits

## 👁️ Final Example

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
printf("%d", numbers[4]);
```

Index 4 → Valid ✅

### Output:

50

## ⚡ Memory Trick

0 → START ✅

size - 1 → END ✅

Outside → DON'T ACCESS! 🛑

---

# QUESTIONS

## 1. Safe Array Loop

Complete the loop condition so that every element is accessed without going out of bounds.

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
for (int i = 0; i < ____; i++) {  
    printf("%d ", numbers[i]);  
}
```

💡 **Hint:** The loop should access indexes 0 through 4.

**Answer:** 5

---

## 2. What is the last valid index?

```
int numbers[5];
```

- A) 3
- B) 4
- C) 5
- D) 6

💡 **Hint:** Last index = size - 1.

📖 **Glossary:** Valid index → An index that refers to an existing element.

✅ **Answer:** B) 4

---

## 3. Which index is invalid?

```
int numbers[4] = {10, 20, 30, 40};
```

- A) 0
- B) 1
- C) 3
- D) 4

💡 **Hint:** Valid indexes are 0 to 3.

📖 **Glossary:** Array bounds → The valid range of indexes.

✓ Answer: D) 4

---

#### 4. How can we safely access an array?

- A) Use any index
- B) Check that the index is within the valid range
- C) Always use index 5
- D) Start indexing from 1

💡 **Hint:** Check the index before accessing.

📖 **Glossary:** Safe access → Accessing only valid array positions.

✓ Answer: B

---

#### 5. Which statement is safest?

```
int numbers[5] = {10, 20, 30, 40, 50};  
int i = 4;
```

- A) `numbers[i]`
- B) `numbers[5]`
- C) `numbers[10]`
- D) `numbers[-1]`

💡 **Hint:** Valid indexes are 0 to 4.

📖 **Glossary:** Index → Position used to access an array element.

✓ Answer: A) `numbers[i]`